

Каждый пункт оценивается в **3 балла**, всего **10 пунктов (30 баллов)**.

Для каждого пункта обязательны:

- реализация “с нуля” (без использования стандартных коллекций Java, если не указано иное);
- набор unit-тестов (JUnit 5);
- покрытие основных и граничных случаев.

Реализовывать все НЕ обязательно! Одно задание == 3 балла

Задания

1. Итератор

Реализовать собственный интерфейс итератора и его применение для обхода коллекции.

Поддержка методов:

- `hasNext()`
 - `next()`
-

2. Список (List)

Реализовать динамический список (аналог ArrayList или LinkedList).

Поддержка:

- добавления
 - удаления
 - получения по индексу
 - изменения элемента
-

3. Хеш-таблица (HashMap)

Реализовать ассоциативный массив.

Требования:

- обработка коллизий (цепочки или открытая адресация)
 - методы: `put` , `get` , `remove`
-

4. Синглтон (ленивая инициализация)

Реализовать паттерн Singleton с ленивой инициализацией.
Обеспечить потокобезопасность.

5. Синглтон (жадная инициализация)

Реализовать Singleton с созданием экземпляра при загрузке класса.

6. Стек (Stack)

Реализовать структуру данных стек.
Методы:

- `push`
 - `pop`
 - `peek`
-

7. Очередь (Queue)

Реализовать очередь (FIFO).
Методы:

- `enqueue`
 - `dequeue`
 - `peek`
-

8. Сортировка

Реализовать два алгоритма сортировки:

- быстрая (QuickSort)
 - слиянием (MergeSort)
-

9. Бинарный поиск

Реализовать алгоритм бинарного поиска для отсортированного массива.

10. Паттерн Фабрика (Factory Method)

Реализовать фабричный метод для создания объектов разных типов.

Пример: создание разных видов фигур / сообщений / транспортов.

Требования к тестам

- Использовать **JUnit 5**
 - Минимум:
 - 3 теста на каждый класс
 - тесты на граничные случаи
 - Проверка:
 - корректности работы
 - обработки ошибок (например, пустые коллекции)
-

- ОБЯЗАТЕЛЬНО ЗНАТЬ КАК ЧТО РАБОТАЕТ!